

where β is an empirically chosen hyperparameter that balances the tradeoff between compression and prediction, consistent with the standard IB formulation. Furthermore, we integrate (6) with the standard fine-tuning objective in (1), yielding the **IB-regularized fine-tuning (IB-FT)**:

$$\underset{\theta, \phi}{\text{minimize}} \quad \ell_{\text{FT}}(\theta) + \alpha \ell_{\text{IB}}(\theta, \phi), \quad (\text{IB-FT})$$

where ℓ_{FT} is the standard cross-entropy loss from (1), and $\alpha > 0$ controls the relative strength of the IB regularization.

Validating the effectiveness of IB through representation analysis. Recall that the motivation for using IB lies in its ability to compress representations while retaining predictive information, thereby reducing the influence of prediction-irrelevant (*i.e.*, spurious) features. This allows differently memorized data to be treated more uniformly during learning, mitigating the memorization barrier that arises from prior differences in memorization strength among code data.

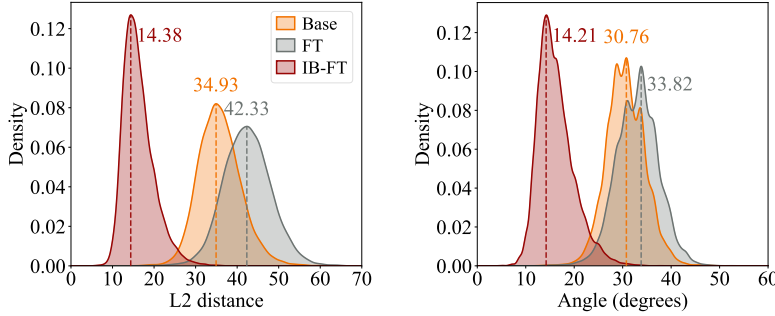


Figure 5: Representation distance and angle analysis with fine-tuning data on OriGen, using the base DeepSeek-Coder-7B-Instruct-v1.5 model and its fine-tuned variants (FT and IB-FT). Hidden representations $h_{\theta}(x)$ are extracted from the 20th layer, and paired samples are drawn between the most-memorized and least-memorized groups (identified by `Min-20% Prob` scores in Fig. 3) to compute (left) their ℓ_2 distance distributions and (right) angular disparity distributions. Dashed vertical lines mark distinct separation patterns under different fine-tuning regimes.

To validate the effectiveness of IB-FT, we analyze the representation geometry between memorized and non-memorized examples. Using `Min-K% Prob` scores, the dataset is partitioned into two subsets (most-memorized group and least-memorized group), and paired samples are drawn to measure ℓ_2 distances and angular disparities between their hidden representations at layer 20. As shown in Fig. 5, standard FT exaggerates the separation between the two groups relative to the base model, creating a barrier that forces the model to treat them in markedly different ways. In contrast, IB-FT substantially compresses this gap, reducing both distance and angular disparity by more than half. These results demonstrate that IB regularization yields more coherent, task-relevant representations across all data groups, thereby enabling fine-tuning to generalize more effectively.

6 Experiments

6.1 Experiment Setup

Datasets and models. Our experiments focus on two established datasets for LLM code fine-tuning: **OriGen** (Cui et al., 2024) for Verilog code generation, and **Evol-CodeAlpaca-V1** (Luo et al., 2024) for multi-language code generation. **OriGen** (Cui et al., 2024) is a Verilog instruction-response corpus with 222,075 examples, used to fine-tune LLMs for register-transfer-level (RTL) code generation. **Evol-CodeAlpaca-V1** (Luo et al., 2024) is a multi-language instruction-to-code dataset of roughly 111,000 examples (covering Python, C++, Java, TypeScript, and Shell).

Following prior work (Cui et al., 2024; Luo et al., 2024), we adopt **DeepSeek-Coder-7B-Instruct-v1.5** (Guo et al., 2024) and **CodeLlama-7B-Instruct** (Roziere et al., 2023) as base models to validate the memorization barrier on the OriGen dataset. For Evol-CodeAlpaca-V1, we use **DeepSeek-Coder-7B-v1.5** (Guo et al., 2024) and **Llama-3-8B** (Grattafiori et al., 2024) as base models. And see Appendix A.1 for more details about model fine-tuning parameters for FT and IB-FT.

Evaluation. For **OriGen**, we evaluate using the **VerilogEval** benchmark (Liu et al., 2023), which comprises two subsets: **Eval-Human** (human-authored problems, *e.g.*, HDLBits and manually transcribed tasks) and **Eval-Machine** (machine-generated prompts produced by LLMs). For **Evol-CodeAlpaca-V1**, we use **HumanEval** (Chen et al., 2021), a standard Python benchmark of 164 programming problems with natural-language descriptions, function signatures, and unit tests for automatic correctness evaluation. For evaluation metric, we introduce $\text{Pass}@k^{(m)}$, which requires at

Table 1: Performance for DeepSeek-Coder-7B-Instruct-v1.5 and CodeLlama-7B-Instruct fine-tuned on OriGen using IB-FT versus conventional fine-tuning (FT). Test-time accuracies are reported on the Eval-Human and Eval-Machine subsets using Pass@ k ($k \in \{1, 5, 10\}$) and the stricter Pass@ $10^{(m)}$ ($m \in \{2, 5, 10\}$), where Pass@ $10^{(m)}$ counts a problem as solved only if at least m of the 10 generated samples pass the unit tests. Best results are shown in **bold**.

Methods	Eval-Human (%)						Eval-Machine (%)					
	Pass@ k $\uparrow$			Pass@ $10^{(m)}$ $\uparrow$			Pass@ k $\uparrow$			Pass@ $10^{(m)}$ $\uparrow$		
	$k = 1$	$k = 5$	$k = 10$	$m = 2$	$m = 5$	$m = 10$	$k = 1$	$k = 5$	$k = 10$	$m = 2$	$m = 5$	$m = 10$
DeepSeek-Coder-7B-Instruct-v1.5												
Base	34.62	42.09	44.23	42.31	36.54	24.36	56.47	64.76	67.13	63.64	58.74	43.66
+ FT	42.63	54.60	57.77	54.49	41.03	27.56	67.83	76.02	76.92	76.23	72.73	49.65
+ IB-FT	51.99	54.86	55.77	54.49	51.92	48.07	75.67	78.51	79.02	78.32	77.62	69.23
CodeLlama-7B-Instruct												
Base	17.56	27.02	29.49	26.28	17.95	4.19	44.76	53.79	55.24	53.85	47.55	27.27
+ FT	46.15	48.47	43.98	48.72	45.13	43.59	66.36	70.53	71.33	70.63	65.03	62.24
+ IB-FT	49.68	52.82	46.81	51.28	50.64	45.51	70.07	72.94	73.47	72.73	70.63	64.33

least m of k samples to succeed, offering a stricter and more reliable measure of generation consistency than standard Pass@ k . More metric details shown in [Appendix A.2](#).

6.2 Experiment Results

Performance overview of IB-regularized fine-tuning (IB-FT). In [Table 1](#) we present performance for LLMs (DeepSeek-Coder-7B-Instruct-v1.5 and CodeLlama-7B-Instruct) fine-tuned on OriGen using FT and our IB-FT, together with each model’s pre-fine-tuning performance. Test-time accuracy is reported on the Eval-Human and Eval-Machine subsets using standard Pass@ k ($k \in \{1, 5, 10\}$) and the stricter Pass@ $10^{(m)}$ (fixed $k = 10$, $m \in \{2, 5, 10\}$).

As [Table 1](#) shows, IB-FT consistently yields the top-rank performance: it outperforms both the base model and conventional FT across nearly all Pass@ k settings, with especially large gains at $k = 1$. For example, on DeepSeek-Coder-7B-Instruct-v1.5 IB-FT achieves 51.99% (Eval-Human) and 75.67% (Eval-Machine) for Pass@1, versus 42.63% and 67.83% for FT. Increasing k (e.g., to $k = 10$) narrows this gap, FT reaches 57.77% Pass@10 for DeepSeek-Coder-7B-Instruct-v1.5, which is better than IB-FT (55.77%). This suggests that some FT gains are realized only via sampling-based evaluation (Pass@ k) rather than as reliable top-1 improvements, consistent with [Fig. 2](#). Crucially, under the stricter Pass@ $10^{(m)}$ criteria ($m > 1$), FT’s scores fall off sharply, indicating that its successes are often isolated single samples rather than consistently reproducible generations. By contrast, IB-FT delivers more robust and reproducible improvements (notably at Pass@1 and Pass@ $10^{(m)}$ for $m > 1$).

To stress, under the stricter Pass@ $10^{(m)}$ metric, IB-FT substantially outperforms conventional FT. For the strictest setting ($m = 10$), which requires all 10 generations to pass, IB-FT improves over FT on DeepSeek-Coder-7B-Instruct-v1.5 by roughly 20% on both Eval-Human (i.e., 48.07% by IB-FT vs. 27.56% by FT) and Eval-Machine (i.e., 69.23% by IB-FT vs. 49.65% by FT). This shows that IB-FT yields stronger generalization and more stable multi-sample performance under demanding criteria, producing more reliable models in practice.

In addition, [Table A1](#) in [Appendix B.1](#) shows that IB-FT also consistently outperforms FT on the Evol-CodeAlpaca-V1 dataset. For example, on HumanEval, IB-FT improves Pass@1 from 56.70% to 61.60% for Llama-3-8B and from 51.28% to 54.57% for DeepSeek-Coder-7B-v1.5. Under the stricter Pass@ $10^{(m)}$ setting, IB-FT further demonstrates stronger robustness, maintaining higher scores than FT across all m values. Also [Fig. A1](#) in [Appendix B.2](#) shows pruning memorized examples, though a natural remedy, is highly sensitive, whereas IB-FT is more robust.

Robustness of IB-FT under different temperature (T) settings. Next, we evaluate code generation after fine-tuning across decoding temperatures $T \in \{0.2, 0.6, 1.0\}$ (the default choice is $T = 0.2$). Varying T probes model robustness to sampling stochasticity and provides a more realistic assessment of fine-tuned performance and stability in practice. This protocol is motivated by the often-overlooked role of temperature in code model evaluation, since higher T yields more diverse generations and can materially affect Pass@ k -style metrics ([Wei et al., 2025](#); [Cui et al., 2024](#)).

[Fig. 6](#) compares DeepSeek-Coder-7B-Instruct-v1.5 fine-tuned with IB-FT, the same model fine-tuned with conventional FT, and the pre-fine-tuned base model across decoding temperatures $T \in \{0.2, 0.6, 1.0\}$. Accuracy is measured by Pass@ $k^{(m)}$ with $m = k \in \{5, 10\}$, i.e., reflecting the all-generation success criterion if all k generated samples pass

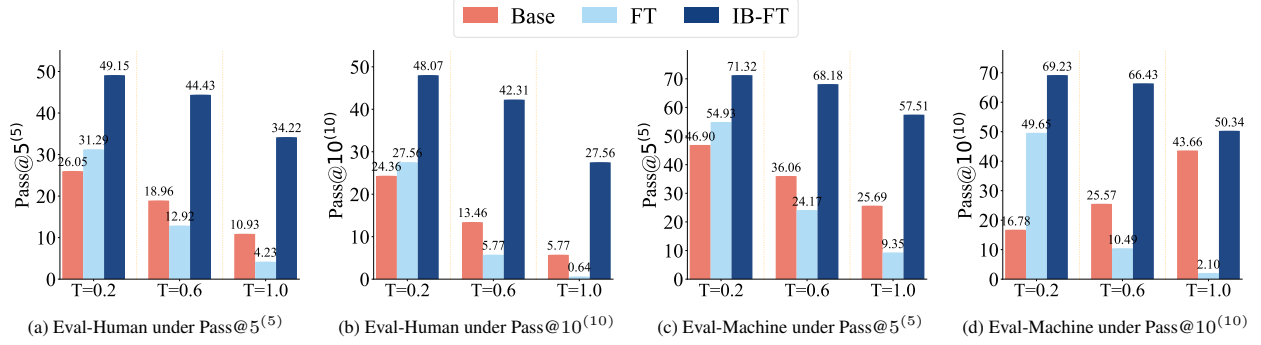


Figure 6: Performance of DeepSeek-Coder-7B-Instruct-v1.5 fine-tuned with IB-FT versus conventional FT and its base model (before fine-tuning) on OriGen across decoding temperatures $T \in \{0.2, 0.6, 1.0\}$. Results are reported on Eval-Human and Eval-Machine using $\text{Pass}@k^{(m)}$ with $m = k \in \{5, 10\}$. Other experimental setups follow Table 1.

the unit tests. As we can see, IB-FT consistently sustains superior performance across all temperatures and benchmarks (Eval-Human and Eval-Machine), substantially outperforming both conventional FT and the base model even at high temperature. For example, on Eval-Human at $T = 1.0$, IB-FT attains 34.22% ($\text{Pass}@5^{(5)}$) versus 4.32% for FT; on Eval-Machine at $T = 1.0$, IB-FT reaches 50.34% while FT collapses to 2.10%. By contrast, FT shows poor robustness to temperature: it only exceeds the base model at the lowest temperature ($T = 0.2$) and suffers large drops as T increases. For instance, on Eval-Human at $T = 0.6$ FT falls to 12.92% ($\text{Pass}@5^{(5)}$) below the base model’s 18.96%, and on Eval-Machine at $T = 1.0$ the base model retains 43.66% ($\text{Pass}@10^{(10)}$) while FT falls to 2.10%. These results indicate that IB-FT mitigates memorization-induced fragility and yields far more stable generalization under decoding stochasticity.

7 Conclusion

We identify a *memorization barrier* in LLM code fine-tuning: pretrained models often already memorize many downstream examples, which traps optimization and limits true adaptation and generalization. To address this, we introduce *IB-regularized fine-tuning* (IB-FT), which applies an information bottleneck (IB) penalty on hidden representations to suppress spurious memorized features while preserving task-relevant signals. Across OriGen and Evol-CodeAlpaca-V1, IB-FT yields consistently stronger and more stable gains, which position IB-FT as a practical, principled method for breaking memorization-driven limits in LLM adaptation.

Acknowledgment

We thank Intel for providing computing resources. The work of C. Wang and S. Liu was partially supported by the National Science Foundation (NSF) CAREER Award IIS-2338068.

References

- Ali Al-Kaswan, Maliheh Izadi, and Arie Van Deursen. Traces of memorisation in large language models for code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–12, 2024.
- Alexander A Alemi, Ian Fischer, Joshua V Dillon, and Kevin Murphy. Deep variational information bottleneck. In *International Conference on Learning Representations*, 2017.
- Olav Beckmann, Alastair Houghton, Michael Mellor, and Paul HJ Kelly. Runtime code generation in c++ as a foundation for domain-specific optimisation. In *Domain-Specific Program Generation: International Seminar, Dagstuhl Castle, Germany, March 23-28, 2003. Revised Papers*, pp. 291–306. Springer, 2004.
- Stella Biderman, Usvsn Prashanth, Lintang Sutawika, Hailey Schoelkopf, Quentin Anthony, Shivanshu Purohit, and Edward Raff. Emergent and predictable memorization in large language models. *Advances in Neural Information Processing Systems*, 36:28072–28090, 2023.
- Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramer, and Chiyuan Zhang. Quantifying memorization across neural language models. *arXiv preprint arXiv:2202.07646*, 2022.